OPTIMIZING A CODE (decreasing time efficiency)

to measure time elapsed

```
import time
start = time.time()
# INCLUDE YOUR CODE HERE
end = time.time()
print(end - start) # in seconds
```

comparative analysis (which is faster):

```
1) statistics.mean(ls) < statistics.fmean(ls)
2) ls[-1] > ls[len(ls)-1]
3) sum = sum + i < sum += i
4) x=x+1 > x=~~x
```

Note that, x + (2's complement of x) = 0
==> x + ((1's complement of x) + 1) = 0
==> x + 1 = − (1's complement of x) = ~~x

5)Try to avoid arithmetic multiplication as and when possible.

n * n will return the same result as that of (m << p) + m,
where n can be represented as 2^p + 1

```
m = 123
val = m * 65
```

It should be written using bitwise left shift as follows.

```
m = 123
val = (m << 6) + m
```

6)Never perform modular division with a power of 2.

m % n will return the same result as that of m & (n-1),
where n is a power of 2.

```
m = 97
val = m % 8
```

It should be written using bitwise AND as follows.

```
m = 97
val = m & 7
```

7) Never perform case insensitive alphabet checking as follows.

```
if ch == 'a' or ch == 'A':
vowel += 1
```

It can be made faster using bitwise OR as follows.

```
ch = ch | 0x20
if ch == 'a':
    vowel += 1
```

8) Lazy evaluation
Typical usage:

```
while i < N and ls[i] >= 0:
...
```

If i >= N, ls[i] is not checked.
This is useful because checking ls[i] >= 0 when i >= N may lead to memory faults.
In such expressions, i >= N serves as a guard condition.

9) Loop jamming
If a single loop contains a lot of operations (it might not fit into the processor's instruction cache), then two separate loops may be faster than a single combined one.
Never use two loops where one will suffice.

```
for i in range(0, 10):
    <Statement 1>
for i in range(0,10):
    <Statement 2>
```

It should be written as follows:

```
for i in range(0,10):
    <Statement 1>
    <Statement 2>
```

10) Loop unrolling

The need for condition check adds some overhead to the program
The loop overhead can be reduced by decreasing the number of iterations and replicating the body of the loop.

```
for i in range(0,100):
    <Statement>
```

The above code can be written as follows:

```
for i in range(0,100,2):
    <Statement>
    <Statement>
```

11) Loop inversion
We should always write count-down-to-zero loops and use simple termination conditions for a better efficiency.

```
fact = 1
i = 1
while i <= n:
    fact *= i
    i += 1
```

The above code can be made efficient by writing as follows:

```
fact = 1
i = n
while i:
    fact *= i
    i -= 1
```

12)

```
print([l for l in ls if l == l[::-1]) < for l in ls:
                                        if l == l[::-1]:
```

Print(l)

Loops versus list comprehensions
List comprehensions are faster than for loops only to create lists.
This is because we are creating a list by appending new elements to it at each iteration.
However, if we want to perform some computations (or call an independent function multiple times) and do not want to create a list, then for loops are faster than list comprehensions.

13)

List comprehensions versus generator expressions
List comprehensions are usually faster than generator expressions as generator expressions create another layer of overhead to store references for the iterator. However, list comprehensions allocate more memory than generator expressions.

14)
Type casting
Avoid type casting wherever possible. Integer and floating point instructions often operate on different registers, so a casting requires a copy
Shorter integer types (char and short) still require the use of a full-sized register, and they need to be padded to 32/64-bits and then converted back to the smaller size before storing back in memory. However, this cost must be weighed against the additional memory cost of a larger data type

15)

Memory organization in lists
Two and higher dimensional arrays are still stored in one dimensional memory. This means ls[i][j] and ls[i][j+1] are adjacent to each other, whereas ls[i][j] and ls[i+1][j] may be arbitrarily far apart.
When modern CPUs load data from main memory into processor cache, they fetch more than a single value. Instead they fetch a block of memory containing the requested data and adjacent data (a cache line). This means after ls[i][j] is in the CPU cache, ls[i][j+1] has a good chance of already being in cache, whereas ls[i+1][j] is still likely to be in the main memory.

PROBLEM SOLVING SKILLS

- Pascal's triangle pattern

IDEA 1
The element in row = i, column = j is obtained by adding the elements in row = i - 1, column = j - 1 and row = i - 1, column = j, except for the first and last column

IDEA 2
The element in row = i, column = j is obtained by computing {i C j} denoting i(C)j

IDEA 3
The element in row = i, column = j is obtained by multiplying the element in row = i, column = j - 1 with (i-j+1)/j, except for the first row and first column.

```
Input: The number of rows row.
for i in range(row):
    for space in range(row-i+1):
        print(' ', end = '')
    for j in range(i+1):
        if j == 0 or i == 0:
            coef = 1
        else:
            coef = (coef * (i-j+1))//j
        print(' ', end = '')
        print(coef, end = '')
    print('')
```

- reverse a list of characters (SWAPPING)

Idea I
Swapping the values between two variables:

```
// Let a = 10, b = 20
t = a // t = 10, a = 10, b = 20
a = b // t = 10, a = 20, b = 20
b = t // t = 10, a = 20, b = 10
// Now a = 20, b = 10
```

- Idea II
Swapping the values between two variables:

```
// Let a = 10, b = 20
a = a * b // a = 200, b = 20
b = a // b // a = 200, b = 10
a = a // b // a = 20, b = 10
// Now a = 20, b = 10
```

Idea III
Swapping the values between two variables:

```
// Let a = 10, b = 20
a = a + b // a = 30, b = 20
b = a - b // a = 30, b = 10
a = a - b // a = 20, b = 10
// Now a = 20, b = 10
```

Idea IV
Swapping the values between two variables:

```
// Let a = 10 (i.e., 00001010), b = 20 (i.e., 00010100)
a = a ^ b // a = 00011110, b = 00010100
b = a ^ b // a = 00011110, b = 00001010
a = a ^ b // a = 00010100, b = 00001010
// Now a = 20 (i.e., 00010100), b = 10 (i.e., 00001010)
```

Recall that, 0 ^ 0 and 1 ^ 1 both returns 0, whereas 0 ^ 1 and 1 ^ 0 both returns 1.

```
Input: The list ls.
i = 0
L = len(ls)
for i in range(L//2):
    t = ls[i]
    ls[i] = ls[L-i-1]
    ls[L-i-1] = t
```

- find out the last repeating character in a string.

- For each character we have to figure out whether it is repeating or not.
- We have to browse the string from the end to the beginning.

Considering time efficiency:
- Looping through the entire string for each character increases the time complexity.

Solution:
- Having two independent loops is better than a pair of nested loops.
- We can use some auxiliary spaces for repetition check.
- What if we compute the frequencies beforehand in an array?
- A character can be used to compute an index of an array.

Note that, Frequency[ASCII('A') - 97] denotes the element Frequency[0].

```
Input: The string str
rc = -1
frequency = [0]*26
for i in range(len(str)):
    frequency[ord(str[i]) - 97] += 1
i -= 1
while i:
    if frequency[ord(str[i]) - 97] > 1:
        rc = i
        break
i -= 1
if rc == -1:
    print('No repeating character')
else:
    print('Last repeating character:', str[rc])
```

- Write a Python program to simulate an environment that generates 1000 random values with the following requirements : Values within [0, 0.5) are generated with a probability 0.7. Values within (0.5, 1] are generated with a probability 0.3. 0.5 is never generated.

- Generate random values with conditional control flows over the values generated.
- Let us simplify the problem to generate random values up to two decimal places only.
- The probability values 0.7 and 0.3 signify 7 out of 10 cases and 3 out of cases, respectively.
- We can control the probability of occurrence with randomization. Generating a random value is nothing but a probabilistic event.
- We can generate random values with randomization.
  1 Generate a random value between [0, 9].
  2 If the random value is less than 7 execute the steps 3-4 and exit, else execute the steps 5-6 and exit.
  3 Generate a random value within [0, 49].
  4 Map it to [0, 0.5)
  5 Generate a random value within [51, 100].
  6 Map it to (0.5, 1]

```
import random
for i in range(1000):
    if random.randint(0, 9) < 7:
        print(random.randint(0, 49)/100)
    else:
        print((51+random.randint(0, 49))/100)
```

- Recall that any arbitrary pair of hands (denoting hour, minute, and second) of a clock forms two possible angles within themselves. Write a program that takes an angle (any one of the possible two) between the other pair of hands (minute and hour) and returns whether there is any valid time satisfying the given angle or not, assuming that the second hand of a clock is residing at 12. Consider that an angle between a pair of hands of a clock will always remain within [0, 2π]. The input is a pair of integers (say m and n) representing the fractional angle between the minute and hour hands (in radian), i.e. the angle is mπ n radian.

When modern CPUs load data from main memory into processor cache, they fetch more than a single value. Instead they fetch a block of memory containing the requested data and adjacent data (a cache line). This means after ls[i][j] is in the CPU cache, ls[i][j+1] has a good chance of already being in cache, whereas ls[i+1][j] is still likely to be in the main memory.

Memory organization in arrays
Accessing data in a more-or-less sequential fashion, as stored in physical memory (in row major fashion), can dramatically speed up the code.

16) Function calls
Move loops inside function calls. Replace the following code
```
Function():
...
for i in range(1,n):
    Function()
```
with this code
```
Function():
    for i in range(1,n):
...
Function()
```
Note: Jumps/branches are expensive and hence should be avoided whenever possible. Note that, function calls require two jumps, in addition to stack memory manipulation

17) Avoiding division
In standard processors, divisions are time-consuming because they take a constant time plus a time for each bit to divide.
```
if (a / b) > c:
    print('OK')
```
It can be efficiently written as follows:
```
if a > (b * c):
    print('OK')
```
Here, the only assumptions are b is non-negative and b * c fits into an integer. The latter one is also safe if b = 0.

18) Optimizing mathematical operations
1)The function sqrt() can often be avoided, especially in comparisons where comparing the value squared gives the same result.
2)In many equations, terms cancel out, either always or in some special cases. Work on them before writing the program.
3)Avoid the use of pow() for computing small integer powers.
```
m = pow(2,4) < m = 2**4 < m = 2*2*2*2
```
4)Instead of repeatedly dividing by x, compute 1/x and multiply accordingly. It is really beneficial if you do more than 3 divides.
```
n = 10
x = 0.5
result = 1.0
for i in range(1,n):
result /= x
```
This can be efficiently done in the following alternative way:
```
n = 10
x = 0.5
result = 1.0
x = result / x
for i in range(1,n):
result *= x
```

- Recall that any arbitrary pair of hands (denoting hour, minute, and second) of a clock forms two possible angles within themselves. Write a program that takes an angle (any one of the possible two) between the other pair of hands (minute and hour) and returns whether there is any valid time satisfying the given angle or not, assuming that the second hand of a clock is residing at 12. Consider that an angle between a pair of hands of a clock will always remain within [0, 2π]. The input is a pair of integers (say m and n) representing the fractional angle between the minute and hour hands (in radian), i.e. the angle is mπ n radian.

- The angles that can be formed by the minute hand are within [0, 2π]. It can have 60 possible values.
- The angles that can be formed by the hour hand are within [0, 2π]. It can have 720 possible values.

The approach
1 Generate all possible angles created by the minute hand.
2 Generate all possible angles created by the hour hand.
3 Take every combination of difference of angles between the minute and hour hands.
4 Verify whether it is the same as m/n or not.

```
Input: The integers m and n
import math
Angle = m/n
flag = 0
for i in range(60):
    AngleMin = 2*i/60
    for j in range(720):
        AngleHour = 2*j/720
        if math.fabs(AngleMin-AngleHour)==Angle or 2-math.fabs(AngleMin-AngleHour)==Angle:
            flag = 1
            break
if flag == 1:
    print('VALID')
else:
    print('INVALID')
```

Efficient version

```
Input: The integers m and n
Angle = m/n
flag = 0
for i in range(60):
    for j in range(720):
        if(n*abs(12*i-j)==360*m or n*(720-abs(12*i-j))==360*m):
            flag = 1
            break
if flag == 1:
    print('VALID')
else:
    print('INVALID')
```